TECHNICAL RESEARCH AND IMPLEMENTATION BLUEPRINT

ML Engineer Challenge: Production-Ready MLOps Solution

A team-readable analysis of the challenge requirements, repository gaps, target architecture, model strategy, runtime optimization, API design, testing, observability and phased delivery plan.

Recommended outcome

Build a versioned three-model computer vision platform: EfficientNet-B0 classification, YOLOX-Tiny detection and OpenCLIP similarity search, served through FastAPI with ONNX/TensorRT optimization and a production data plane.

Document	Scope	Status
Audience	Engineering, ML, platform, QA and technical leadership	Team review
Research date	17 July 2026	Current as reviewed
Primary source	Applied Computing ML Engineer Challenge repository	Direct repository review
Decision level	Production blueprint; no benchmark numbers are fabricated	Ready for Phase 1

Prepared for collaborative reading, implementation planning and design review.

Contents

Major sections and implementation topics covered in this research document.

1. Executive summary

The challenge is primarily an ML systems engineering assessment. Model accuracy matters, but the evaluation gives equal strategic weight to production readiness, reliability and operational quality.

Core recommendation

Treat the solution as one product with an offline model lifecycle and an online inference platform. Avoid three disconnected model notebooks. Complete one classification path end-to-end first, then generalize the runtime and add detection and similarity.

What the team will build

- Three versioned computer vision tasks: classification, object detection and image similarity search.
- One runtime abstraction supporting PyTorch, ONNX Runtime and TensorRT backends.
- INT8 artifacts produced with representative calibration data and guarded by task-specific accuracy thresholds.
- FastAPI synchronous endpoints plus Celery background jobs for large batches.
- Redis for cache, distributed rate limiting and the Celery broker; PostgreSQL for durable metadata and pgvector similarity search.
- Nginx, Prometheus, Grafana, structured logs, health probes, GitHub Actions and containerized local/production profiles.

How the challenge is weighted

Evaluation area	Weight	Implication for implementation
Technical excellence	30%	Architecture, code quality, model optimization, scalability and reliability must be demonstrable.
Production readiness	30%	Containers, monitoring, error handling and operational resilience are first-class deliverables.
ML engineering	30%	Training, conversion, quantization, benchmarking, validation and MLOps depth must be reproducible.
Communication	10%	README, model cards, API examples, assumptions and design decisions must be concise and evidence-backed.

Source: challenge README [S1].

2. Challenge interpretation and confirmed ambiguities

The repository contains internal contradictions. A strong submission should not hide them; it should document a reasonable interpretation and design the implementation so the assumptions are reversible.

Observed ambiguity	Evidence	Documented decision
Three models are requested, but only classification and detection are listed in the model bullet list.	The overview names similarity, and the supplied load test calls /api/v1/similarity [S1, S5].	The third model is image similarity using an OpenCLIP image encoder.
Classification is associated with both CIFAR-100 and Tiny ImageNet.	The same requirement section mentions both datasets [S1].	Build a dataset adapter supporting both. The primary fine-tuned artifact uses Tiny ImageNet; CIFAR-100 is an optional secondary experiment.
The dataset script path changes within the README.	The actual file is scripts/download_datasets.py [S2].	Use the real path and correct the project documentation.
Authentication is mandatory but no auth contract is provided.	The load test invents /auth/login, while the API requirements do not [S1, S5].	Use hashed API keys with configurable user tiers. Update the load test accordingly.
TensorRT output is requested without defining GPU infrastructure.	TensorRT engines are target/version sensitive [S12].	Pin a CUDA/TensorRT/GPU test environment and never report unmeasured TensorRT results.

Required assumption record

Create docs/assumptions.md and record each ambiguity, selected interpretation, rationale, alternative and impact. This demonstrates engineering judgment and makes evaluator discussions straightforward.

Decision boundary

The team should implement every explicitly required capability. Only optional polishing - such as OpenTelemetry, Kubernetes manifests or a hosted model registry UI - should be deferred when time is limited.

3. Repository assessment

The repository is a collection of helper scripts and project-generation hints. It is not a safe production base without substantial corrections.

Area	Finding	Production response
Serving	The sample uses Flask, loads ResNet globally and lacks comprehensive validation, auth, versioning and controlled concurrency [S9].	Replace with FastAPI routers, services, lifespan loading and runtime adapters.
Tiny ImageNet	The validation split is treated as ImageFolder even though labels are defined in val_annotations.txt [S3].	Implement a custom validation Dataset or reorganize the split deterministically.

Area	Finding	Production response
Dataset download	HTTP downloads, incomplete integrity checks, weak timeout/error handling and unrestricted archive extraction [S2].	Add HTTPS where available, retries, checksums, safe extraction and explicit resulting paths.
Model registry	A local JSON file is not replica-safe and supports only .pth/.pkl loading [S4].	Use durable model metadata plus immutable artifact manifests and checksums.
Configuration	Generated defaults include placeholder secrets and simplistic dynamic quantization [S8].	Use Pydantic Settings, environment validation and separate CPU/GPU runtime profiles.
Logging	Correlation state is held in a mutable global object [S6].	Use contextvars so concurrent requests do not leak request IDs.
Health checks	Credentials and endpoints are hardcoded; file existence is treated as model readiness [S7].	Separate liveness and readiness and perform runtime smoke inference.
Monitoring	The generated Prometheus config scrapes raw Redis/Postgres ports [S8].	Use application metrics and dedicated Redis/Postgres exporters.
Load testing	The supplied scenarios are useful but target endpoints and auth that do not consistently exist [S5].	Align tests to the final OpenAPI contract and record SLO pass/fail thresholds.

4. Target system architecture

The architecture separates request handling, model execution, durable state and background work so each can be tested, monitored and scaled independently.

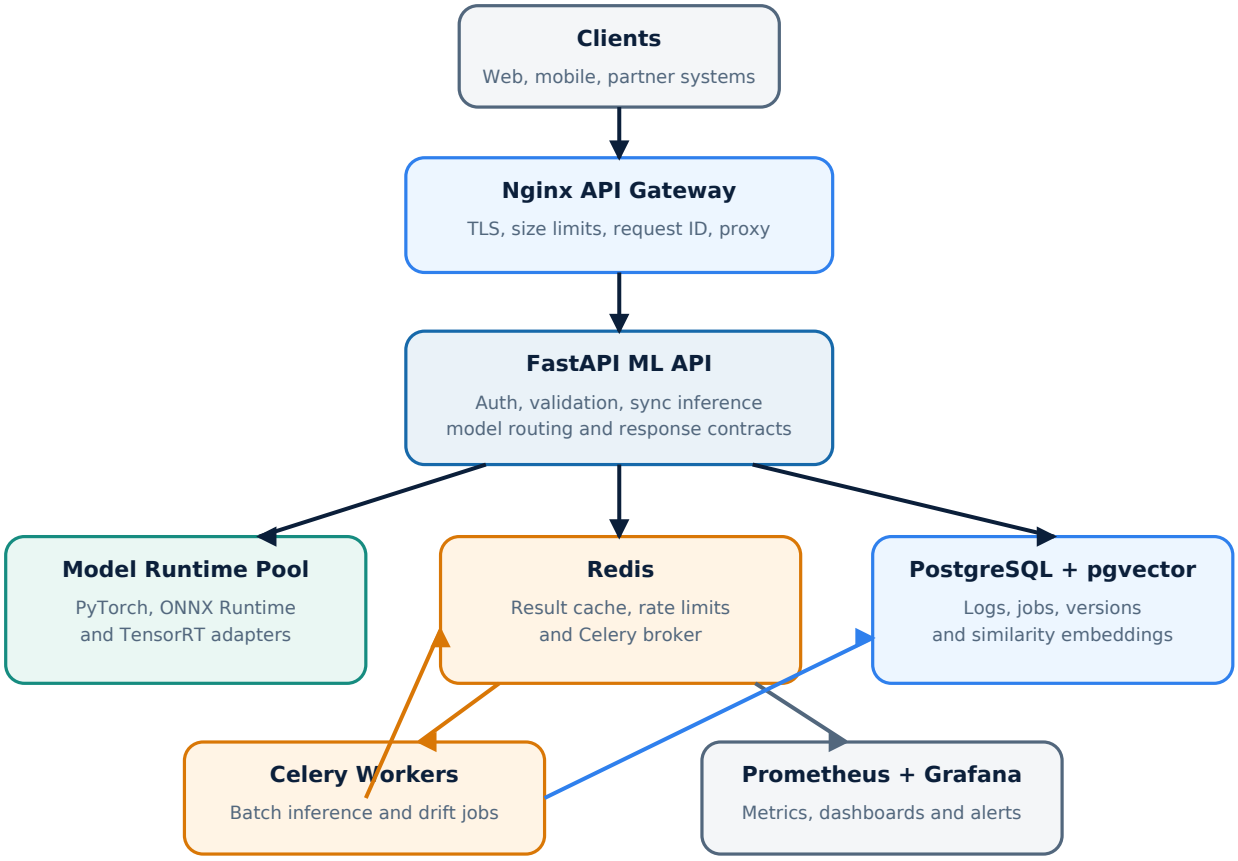

Figure 1. Recommended production architecture and primary data flows.

Component	Primary responsibility	Scaling / failure behavior
Nginx gateway	TLS, request-size limits, request ID propagation, coarse IP protection and proxying.	Stateless; scale separately or place behind a managed load balancer.
FastAPI ML API	Authentication, image validation, synchronous inference, routing and response contracts.	CPU replicas scale horizontally. GPU deployments typically use one process per GPU to avoid duplicated VRAM.
Runtime pool	Load and execute PyTorch, ONNX or TensorRT artifacts behind a common interface.	Use bounded concurrency/semaphores; fall back only to the same semantic model version.
Redis	Cache, distributed token bucket and Celery broker.	Cache failure should not block sync inference; broker failure makes batch unavailable.
PostgreSQL	API keys, jobs, model versions, inference metadata, experiments and drift results.	Durable state with migrations, connection pooling and retention policies.
pgvector	Store normalized OpenCLIP embeddings and execute cosine nearest-neighbor search.	Use versioned namespaces and HNSW after validating recall/latency trade-offs [S15].

Component	Primary responsibility	Scaling / failure behavior
Celery workers	Batch inference, indexing and scheduled drift checks.	CPU and GPU queues use different concurrency; tasks are idempotent [S14].
Prometheus/Grafana	Operational metrics, dashboards and alerts.	Avoid user IDs and other unbounded metric labels [S16].

Synchronous request flow

1. Nginx validates coarse request constraints and establishes a correlation ID.
2. FastAPI authenticates the API key, applies the Redis-backed tier limit and decodes the image safely.
3. The router resolves the requested task, champion model version and preferred runtime backend.
4. A content-aware cache key is checked. On miss, the bounded runtime executes inference.
5. Postprocessing creates a versioned response; inference metadata is recorded asynchronously and the cache is populated.

Batch request flow

Large inputs are written to a blob-store abstraction; only object URIs and job identifiers enter Redis/Celery. The worker updates durable progress in PostgreSQL, retries only safe failures and writes the final result reference. Redis must not become a storage layer for large image payloads.

5. Model strategy

The selected models balance accuracy, exportability, quantization support, inference cost and production licensing. The classifier is the only model that must be fine-tuned by the explicit wording of the challenge.

Task	Selected model	Training / data	Primary metrics	Reason
Classification	EfficientNet-B0	ImageNet-pretrained; fine-tune on Tiny ImageNet. Config also supports CIFAR-100.	Top-1, Top-5, macro F1, calibration error	Strong speed/accuracy balance; manageable artifact size; deployable across backends.
Detection	YOLOX-Tiny	Pretrained COCO weights; validate on deterministic COCO subset.	mAP50:95, mAP50, precision, recall	Small production footprint, established ONNX/TensorRT path and permissive code license.
Similarity	OpenCLIP ViT-B/32 image encoder	Pretrained encoder; index a versioned image gallery in pgvector.	Recall@K, MRR, embedding parity, retrieval latency	High-quality general image embeddings and a clear vector-search use case.

Classification training design

- Pretrained EfficientNet-B0 with a 200-class classification head for Tiny ImageNet.
- Automatic mixed precision, AdamW, gradient clipping, warm-up plus cosine scheduling, label smoothing and early stopping.
- Custom augmentation: random resized crop, horizontal flip, color jitter or RandAugment, random erasing and normalization.
- Fixed seeds, recorded split manifest, deterministic validation, best-checkpoint selection and per-class analysis.
- A correct Tiny ImageNet validation loader based on val_annotations.txt, replacing the starter implementation [S3].

Detection design

Export raw boxes and scores where feasible and keep confidence filtering and NMS in shared versioned postprocessing. This reduces backend-specific behavior and makes PyTorch/ONNX/TensorRT parity tests meaningful. Calibration images must represent the detector's actual operating distribution.

Similarity design

L2-normalize embeddings, store the embedding model version with every vector and query using cosine distance. A new encoder version requires a new namespace or controlled re-indexing. Never compare vectors from incompatible embedding versions silently.

Licensing check

Before production release, record model code license and pretrained-weight terms in every model card. A technically working artifact is not automatically approved for commercial use.

6. Offline model lifecycle and optimization

Every promoted model must move through a repeatable pipeline. Conversion success alone is not enough: parity, quality and performance gates are mandatory.

Each transition is blocked by parity, accuracy, reliability and performance gates.

Figure 2. Artifact lifecycle and promotion gates.

Produced artifacts

Artifact	Purpose	Required validation
PyTorch FP32	Training reference and accuracy baseline.	Dataset metrics and deterministic smoke inference.
PyTorch AMP	GPU baseline with reduced precision.	Output parity, NaN/Inf checks and latency measurement.
ONNX FP32	Portable graph for CPU/GPU inference.	ONNX checker, runtime parity and dynamic/fixed shape contract.
ONNX INT8	Primary portable quantized artifact.	Representative calibration and task-specific accuracy gates [S11].
TensorRT FP16	High-performance NVIDIA GPU artifact.	Target-GPU build, parity, warm-up and latency/throughput benchmark.
TensorRT INT8	Lowest-latency target where quality permits.	Calibration/QAT validation and hardware-specific compatibility record [S12].

Quantization policy

For convolution-heavy models, static post-training quantization with representative data is the default. ONNX Runtime calculates activation parameters from calibration data and supports MinMax, entropy and percentile strategies [S11]. Transformer linear layers can also use current TorchAO INT8 workflows [S17], but one approach should not be assumed to optimize every operation in every model.

If INT8 exceeds an accuracy-loss gate, the response is one of: improve calibration coverage, retain sensitive layers at higher precision, use quantization-aware training, or reject INT8 promotion for that artifact. The challenge requires INT8 work, but production quality must remain explicit.

Benchmark method

- Record exact CPU/GPU, RAM/VRAM, OS, CUDA, TensorRT, framework and runtime versions.
- Use warm-up iterations and enough measured runs for stable p50, p95 and p99 latency.
- Synchronize CUDA around GPU timings and separate model-only latency from end-to-end API latency.
- Measure throughput, memory, artifact size, preprocessing, postprocessing and concurrency behavior.
- Save raw JSON/CSV plus a generated Markdown report; never publish only a single average.

- Re-run the baseline and candidate on the same hardware and corpus. NVIDIA recommends measure, optimize and remeasure [S13].

Initial quality gates

Task	Proposed maximum quality loss vs FP32	Operational target
Classification	Top-1 drop no greater than 1 percentage point.	Single-image p95 below one second; measurable size or latency benefit.
Detection	mAP50:95 drop no greater than 1-2 absolute points.	Stable output schema and bounded detection count.
Similarity	Recall@K drop no greater than 1 point; bounded cosine drift.	Predictable retrieval p95 and version-consistent index.

These are proposed acceptance gates, not claimed benchmark results. Actual values must be measured and documented.

7. Artifact registry and model versioning

Runtime reliability depends on immutable artifacts, explicit preprocessing contracts and traceable promotion decisions.

```
artifacts/
  classification/efficientnet_b0/1.0.0/
    manifest.json
    model.pt
    model.onnx
    model.int8.onnx
    model.fp16.plan
    model.int8.plan
    labels.json
    preprocessing.json
    benchmark.json
    MODEL_CARD.md
```

Manifest fields

- Model name, semantic version, task, status and champion/challenger alias.
- Git commit, dataset/split version, training configuration and calibration corpus.
- Input/output signatures, class labels, preprocessing version and postprocessing version.
- Artifact URIs and SHA-256 checksums for every backend.
- Framework, CUDA, TensorRT, hardware compatibility and benchmark environment.
- Validation metrics, known limitations, license details and promotion approvals.

TensorRT portability

Do not treat one .plan file as universally deployable. TensorRT engine compatibility depends on platform, runtime version and GPU settings. Build on the target environment or use an explicitly tested compatibility mode [S12].

For the challenge, model metadata can live in PostgreSQL while artifacts live in a mounted or object-store-backed location. An offline experiment tracker such as MLflow can be added without making online serving depend on the tracking server.

8. Production API contract

The API contract should be explicit, versioned and identical across runtime backends. FastAPI lifespan should load shared models just before the application begins serving requests and release them during shutdown [S10].

Method	Endpoint	Behavior
POST	/api/v1/classify	Validated synchronous image classification.
POST	/api/v1/detect	Validated object detection with bounded confidence/IoU parameters.
POST	/api/v1/similarity	Create an embedding and retrieve version-compatible similar images.
POST	/api/v1/batch	Persist inputs and enqueue asynchronous processing; return HTTP 202.
GET	/api/v1/batch/{job_id}	Return job status, progress, errors and result reference.

Method	Endpoint	Behavior
GET	/api/v1/models	List task, model, version, runtime, health and promotion metadata.
GET	/api/v1/health/live	Process liveness only.
GET	/api/v1/health/ready	Models and mandatory dependencies are ready.
GET	/api/v1/health	Challenge-compatible aggregate health route.
GET	/api/v1/metrics	Prometheus exposition endpoint.

Response envelope

```
{
  "request_id": "uuid",
  "task": "classification",
  "model": "efficientnet_b0",
  "model_version": "1.0.0",
  "backend": "tensorrt-int8",
  "cached": false,
  "latency_ms": 18.4,
  "predictions": []
}
```

Input controls

- Prefer multipart image uploads over base64 for normal requests.
- Validate decoded content, not only MIME type or extension.
- Enforce file-byte, pixel-count, dimension, batch-count and total-request limits.
- Protect against decompression bombs, malformed images and unsafe archives.
- Normalize EXIF orientation and color mode and bound top_k, confidence and IoU inputs.
- Never expose internal exception traces, model paths or secrets in client responses.

9. Data, cache and asynchronous processing

Durable business state belongs in PostgreSQL; ephemeral coordination and cache state belong in Redis; large binaries belong in a blob store or shared development volume.

Core database entities

Entity	Key fields	Purpose
api_keys	id, key_hash, tier, active, created_at	Authentication and tier configuration without storing plaintext keys.
model_versions	task, name, version, status, manifest_uri, checksum	Runtime discovery, champion/challenger state and traceability.
inference_events	request_id, task, version, backend, latency, status, cached	Operational history and model monitoring without requiring raw-image retention.
batch_jobs	job_id, status, total, completed, failed, timestamps	Durable job progress and idempotency.
experiments	experiment_id, variant weights, versions, status	A/B routing configuration and audit history.
drift_windows	model_version, window, test, statistic, threshold, flagged	Periodic drift evidence and alert traceability.
image_embeddings	image_id, embedding_version, vector, metadata	Versioned similarity gallery and nearest-neighbor retrieval.

Cache key

```
SHA256(image_bytes + task + model_version + preprocessing_version + normalized_parameters)
```

Cache entries contain final response data and a bounded TTL. They do not contain plaintext API keys or unnecessary raw images. A model or preprocessing promotion naturally creates a different key namespace.

Celery reliability rules

- Tasks are idempotent using job_id and item_id; duplicate delivery must not duplicate durable side effects.
- Retries use exponential backoff and classify transient versus permanent errors.
- Late acknowledgement is enabled only for idempotent jobs; Celery notes that redelivery may occur [S14].
- CPU and GPU queues are separate. Start GPU concurrency at one worker per GPU and tune from measured VRAM/throughput.
- Worker progress is written to PostgreSQL; Redis is not the source of truth for completed results.
- Cancellation is cooperative. Revoking a queued task does not guarantee a running inference is immediately terminated.

Graceful degradation

Failure	Expected behavior
TensorRT runtime unavailable	Use the same model/version through ONNX if explicitly configured; record fallback_backend.
Redis cache unavailable	Continue synchronous inference without cache; use conservative local rate protection and emit an alert.
Redis broker unavailable	Return 503 for batch submission; keep synchronous inference available if independent dependencies are healthy.
PostgreSQL logging unavailable	Do not discard a valid prediction solely because audit logging failed; increment a failure metric and apply a bounded retry/buffer policy.
All model backends unavailable	Fail readiness and return a stable 503 error envelope. Never silently switch to a semantically different model.

10. Authentication, A/B testing and drift detection

These features must be deterministic and auditable. Random routing without assignment stability or drift alerts without effect-size controls would appear complete while remaining operationally weak.

API-key authentication and rate limiting

- Accept X-API-Key; store a salted cryptographic hash, never the plaintext secret.
- Cache key metadata in Redis and associate each identity with free, pro or internal tiers.
- Use an atomic Redis Lua token bucket keyed by API identity and endpoint class.
- Return HTTP 429 with Retry-After; add coarse Nginx IP limits as pre-authentication protection.
- Keep quotas configurable in data/configuration rather than embedding them in middleware code.

A/B assignment

```
variant_bucket = hash(api_key_id + subject_id + experiment_id) % 100
```

The deterministic hash gives a subject a stable variant. Log experiment ID, variant, model version, backend, latency, error and available delayed ground truth. Online traffic without labels can compare reliability and latency, but cannot alone prove accuracy improvement.

Drift monitoring

Signal	Suggested method	Control
RGB/brightness/aspect ratio	KS test plus effect size	Minimum sample window; alert only when statistical and practical thresholds are exceeded.
Predicted class/confidence	Jensen-Shannon divergence and histogram deltas	Compare per model version and meaningful traffic slice.
Embedding distribution	PCA projections plus KS; MMD for multivariate shift	Sample with retention limits; never mix encoder versions.
Detection outputs	Object count, score and box-area distributions	Separate data drift from runtime regressions and threshold changes.

Celery Beat can execute scheduled drift jobs, persist results in drift_windows and publish bounded-cardinality Prometheus signals. Raw customer images should not be retained by default; monitoring can rely on sampled derived features and configurable privacy controls.

11. Observability and operational targets

Observability covers requests, dependencies, model execution and data quality. Correlation IDs belong in logs and traces, not as high-cardinality Prometheus labels.

Category	Examples
API	request_total, request_latency_seconds, response_status_total, inflight_requests
Model	model_inference_seconds, model_errors_total, fallback_total, model_loaded, runtime_queue_depth

Category	Examples
Cache and batch	cache_hit_ratio, redis_errors_total, celery_queue_depth, job_duration_seconds, job_failures_total
Resources	process memory, CPU, GPU utilization, VRAM, container restarts
Quality	drift_score, drift_flag, prediction_confidence summary, quantized parity failures

Prometheus warns that every label combination creates a new time series. Do not use request IDs, user IDs, emails or image hashes as labels [S16]. Put them in structured logs instead.

Initial SLOs to validate

- Availability: 99.9% for synchronous API during the evaluated production window.
- Latency: p95 below one second per single-image request; task-specific tighter targets set after baseline measurement.
- Error rate: below 1% excluding validated client errors.
- Batch reliability: durable terminal state for every accepted job and no duplicate committed item result.
- Readiness: false when the champion model cannot pass a smoke inference or mandatory state dependencies are unavailable.

12. Testing and security strategy

Coverage percentage is only one signal. The test suite must prove runtime parity, dependency behavior, model quality, container health and overload characteristics.

Test layer	Examples	Execution
Unit	Image validators, preprocessing, cache keys, auth, rate limits, routing and postprocessing.	Every pull request; external dependencies mocked only at boundaries.
Integration	PostgreSQL migrations, Redis Lua limiter, cache behavior, Celery task lifecycle and pgvector queries.	Pull request using disposable services/test containers.
Runtime parity	PyTorch vs ONNX vs TensorRT outputs on a fixed golden corpus.	CPU on pull requests; GPU on self-hosted/scheduled runners.
Model regression	Accuracy, mAP, Recall@K and calibration/quantization thresholds.	Promotion workflow and scheduled validation.
End-to-end	Gateway through API, database, queue, worker and result retrieval.	Docker Compose in CI/staging.
Performance	Warm load, peak load, stress, endurance, memory leaks and queue saturation.	Locust runs with versioned reports and pass/fail SLOs.
Security	Malformed files, oversized inputs, archive traversal, secrets, dependencies and container CVEs.	Pull request and release gates.

Coverage targets

Aim above 90% for deterministic service and utility code, while enforcing at least 85% on critical application paths as required by the challenge. Model-framework internals and generated migration files should not distort the number. A high percentage cannot substitute for integration and model-regression tests.

Security controls

- Non-root containers, read-only root filesystem where possible, dropped Linux capabilities and explicit writable volumes.
- Pinned dependencies and base-image digests; automated dependency and image scanning; SBOM on release.
- No secrets in source, compose files, images, logs or model manifests. Load them from environment/secret stores and validate at startup.
- Upload limits at Nginx and FastAPI; safe image decoding; no unrestricted pickle from untrusted sources.
- Database least privilege, parameterized queries, migration controls, retention policies and audit events for key/model changes.

13. Containerization, deployment and CI/CD

Use one repository and multi-stage Dockerfiles with targets for the API and worker. Keep CPU and NVIDIA/TensorRT dependency sets separate to avoid forcing every local developer and CI job to build a massive GPU image.

Profile	Services	Purpose
Local CPU	Nginx, FastAPI/ONNX CPU, worker, Redis, PostgreSQL+pgvector, exporters, Prometheus, Grafana	Developer setup and most integration tests.
GPU override	TensorRT-enabled API/worker with NVIDIA device reservation	Conversion validation, GPU parity and benchmark execution.
Production Compose	Pinned images, health checks, resource limits, secrets, persistent volumes and separate networks	Challenge deliverable and small production-like deployment.
Future orchestrator	Kubernetes/ECS/managed container platform	Horizontal scaling, rollout policy and managed storage after the challenge.

GitHub Actions pipeline

Trigger	Pipeline stages
Pull request	Ruff/format, type checks, unit tests, coverage, integration services, ONNX smoke test, security scans and CPU image build.
Main branch	All PR gates plus end-to-end Compose smoke tests and immutable image publication by commit SHA.
Manual/scheduled GPU	Export, TensorRT build, parity, calibration validation and reproducible performance benchmarks.
Model promotion	Validate model card/manifest/checksums, compare regression gates, register challenger, canary/A-B and approve champion.
Release	SBOM, image signing if available, tagged artifacts, deployment, readiness smoke test and rollback record.

Cost and reproducibility

Do not train full models in every pull request. PRs use tiny deterministic fixtures and lightweight ONNX artifacts. Full training and GPU benchmarks run in explicit workflows with recorded hardware and dataset versions.

14. Recommended repository structure

The layout follows the challenge's expected top-level folders while separating API contracts, model code, runtime code, durable state and operational assets.

```
ml-engineer-challenge/
  api/
    main.py
    routers/  middleware/  schemas/  dependencies/
  models/
    classification/  detection/  similarity/
    runtimes/  export/  quantization/  registry/
  workers/
  db/
    models.py  repositories/  migrations/
  benchmarks/
  configs/
  monitoring/
  docker/
  nginx/
  scripts/
  tests/
    unit/  integration/  e2e/
    model_regression/  performance/
  docs/
  .github/workflows/
  pyproject.toml
  requirements.txt
  docker-compose.yml
  docker-compose.prod.yml
```

Key documentation deliverables

- README.md: quick start, architecture, API examples, benchmark summary and limitations.
- docs/assumptions.md: repository ambiguities and reversible decisions.
- docs/architecture.md: component responsibilities, request flows, scaling and failure modes.
- docs/security.md: threat model, upload controls, auth, secrets and dependency policy.
- docs/benchmarking.md: corpus, commands, hardware, warm-up, timing and quality gates.
- MODEL_CARD.md per model version: data, metrics, intended use, limitations, license and ethical/operational considerations.
- OpenAPI and Postman collection: runnable endpoint examples, auth and error envelopes.

15. Phased implementation plan

Phase	Primary work	Exit gate
1. Foundation	Scaffold project, dependency lock, settings, migrations, assumptions, CI skeleton.	Fresh checkout installs; lint/tests run; configuration fails clearly when required values are absent.
2. Classification vertical slice	Correct Tiny ImageNet pipeline, fine-tuning, model card, ONNX export and /classify.	Golden images pass; FP32 metrics recorded; one end-to-end endpoint works through container.
3. Runtime abstraction	PyTorch, ONNX and TensorRT adapters; manifest loader; concurrency controls.	Same API response contract; backend parity tests pass; startup/readiness behavior verified.
4. Detection and similarity	YOLOX inference/export; OpenCLIP embeddings; pgvector index; endpoints.	Task metrics and golden-corpus tests pass; no cross-version embedding mix.
5. Optimization	INT8 calibration, TensorRT build, benchmark runner and reports.	Every promoted artifact has reproducible performance and quality evidence.
6. Production data plane	Redis cache/limiter, PostgreSQL logs, API keys, Celery batch and blob-store abstraction.	Duplicate delivery safe; batch polling works; cache/dependency failure tests pass.
7. MLOps validation	Registry state, A/B assignment, drift jobs and promotion gates.	Stable assignment, audit history, drift fixtures and champion rollback demonstrated.
8. Hardening and submission	Load tests, security, observability, Compose production profile and complete docs.	85%+ critical coverage, SLO report, dashboards, clean setup guide and evaluator demo script.

Recommended build order principle

Build one thin vertical path first

Finish classification from correct data loading through training, export, runtime parity, FastAPI and container smoke testing before expanding horizontally. This exposes the riskiest integration problems while the codebase is still small.

16. Risks, trade-offs and mitigation

The following risks are likely to affect schedule or production credibility. They should be visible in planning rather than discovered at final integration.

Risk	Impact	Mitigation
No suitable NVIDIA runner	TensorRT artifacts and benchmarks cannot be honestly completed.	Secure the target GPU early; continue ONNX work locally; pin and record the environment.
INT8 quality regression	Fails model-quality expectations despite faster inference.	Representative calibration, per-layer precision controls, QAT fallback and explicit reject gate.
ONNX unsupported operations	Export or parity failures, especially detector postprocessing.	Export raw outputs, keep shared postprocessing outside graph and use early vertical-slice experiments.
Tiny ImageNet validation bug	Misleading accuracy and invalid model card.	Custom val_annotations parser, dataset integrity tests and manual sample verification.
GPU over-concurrency	VRAM exhaustion, latency spikes and worker crashes.	One process/concurrency per GPU initially; bounded queues and measured tuning.
Redis treated as durable storage	Lost jobs/results and excessive memory usage.	PostgreSQL is source of truth; blob storage holds binaries; Redis stores coordination/cache only.
High metric cardinality	Prometheus memory/cost explosion.	Bounded task/model/backend/status labels; request and user identifiers remain in logs [S16].
Scope overload	Many incomplete features with weak evidence.	Use phase exit gates; fully implement explicit requirements before optional infrastructure.
Model/weight licensing	Technically valid solution may be unusable commercially.	Record code and weight terms in model cards before selection/promotion.

What the team should avoid

- Do not submit only notebooks or screenshots of successful predictions.
- Do not claim sub-second inference or TensorRT speedups without a recorded benchmark environment.
- Do not use dynamic quantization as a universal claim of INT8 coverage for convolutional models.
- Do not load a separate GPU model copy in multiple Uvicorn workers without measuring VRAM impact.
- Do not store image bytes in Celery messages or raw images indefinitely for drift monitoring.
- Do not silently fall back to another model version or compare incompatible similarity embeddings.
- Do not make online serving depend on an experiment-tracking UI being available.

17. Final recommendation and definition of done

Proceed with the proposed architecture, beginning with Phase 1 and the classification vertical slice. The solution is complete only when another engineer can clone the repository, start the documented environment, run tests, call all endpoints and inspect reproducible model and operational evidence.

Definition of done

1. Three production endpoints and a working asynchronous batch lifecycle.
2. Versioned PyTorch, ONNX and TensorRT artifacts for every model, including INT8 work and measured quality deltas.
3. Correct Tiny ImageNet training/validation and recorded classification metrics.
4. COCO-subset detection metrics and versioned similarity retrieval metrics.
5. Runtime parity, model-regression, integration, end-to-end and load-test evidence.
6. API-key auth, tier-aware rate limiting, cache, durable jobs and explicit degradation behavior.
7. A/B assignment, drift monitoring, Prometheus metrics, Grafana dashboards and actionable alerts.
8. Secure multi-stage containers, local and production Compose profiles and CI/CD workflows.
9. Complete README, assumptions, architecture, API, benchmark and model-card documentation.

Bottom line

The strongest submission will demonstrate engineering judgment: explicit assumptions, correct data handling, measured optimization, reliable asynchronous work, operational visibility and clear rollback paths. The platform should remain understandable even when a particular model or runtime is replaced.

18. Research sources

Repository evidence and primary technical documentation reviewed for this blueprint. Accessed 17 July 2026.

S1 - Applied Computing - ML Engineer Challenge README
https://github.com/appliedcomputingtech/ML-Engineer-Challenge/blob/main/README.md

S2 - Challenge dataset downloader
https://github.com/appliedcomputingtech/ML-Engineer-Challenge/blob/main/scripts/download_datasets.py

S3 - Challenge Tiny ImageNet data loader
https://github.com/appliedcomputingtech/ML-Engineer-Challenge/blob/main/scripts/tiny_imagenet_dataloader.py

S4 - Challenge model registry helper
https://github.com/appliedcomputingtech/ML-Engineer-Challenge/blob/main/scripts/models/model_registry.py

S5 - Challenge Locust load tests
https://github.com/appliedcomputingtech/ML-Engineer-Challenge/blob/main/scripts/testing/load_tests.py

S6 - Challenge logging configuration
https://github.com/appliedcomputingtech/ML-Engineer-Challenge/blob/main/scripts/utils/logging_config.py

S7 - Challenge container health checker
https://github.com/appliedcomputingtech/ML-Engineer-Challenge/blob/main/scripts/docker/health_check.py

S8 - Challenge project setup generator
https://github.com/appliedcomputingtech/ML-Engineer-Challenge/blob/main/scripts/setup/setup_project.py

S9 - Challenge sample serving script
https://github.com/appliedcomputingtech/ML-Engineer-Challenge/blob/main/scripts/sample_serving_script.py

S10 - FastAPI lifespan events
https://fastapi.tiangolo.com/advanced/events/

S11 - ONNX Runtime model quantization
https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html

S12 - NVIDIA TensorRT engine compatibility
https://docs.nvidia.com/deeplearning/tensorrt/latest/inference-library/engine-compatibility.html

S13 - NVIDIA TensorRT performance best practices
https://docs.nvidia.com/deeplearning/tensorrt/latest/performance/best-practices.html

S14 - Celery task reliability and idempotency
https://docs.celeryq.dev/en/stable/userguide/tasks.html

S15 - pgvector vector search and HNSW indexing
https://github.com/pgvector/pgvector

S16 - Prometheus instrumentation practices
https://prometheus.io/docs/practices/instrumentation/

S17 - TorchAO quantization API reference
https://docs.pytorch.org/ao/stable/api_reference/api_ref_quantization.html

Research note: repository-specific findings are based on direct inspection of the listed source files. Proposed thresholds, component choices and implementation phases are design recommendations and must be validated against available hardware, delivery time and measured model behavior.